Reproducing and Extending Contrastive Decoding: From Text to Music Generation

Yoongyu Heo, Winnie Liu, Bradley Lasker, Timothy Nguyen

GitHub Link: jygheo/Contrastive-Decoding

Introduction

Open-ended text generation presents some core issues: search-based decoding, such as greedy search, produces repetitive, low-diversity text, while sampling-based methods can yield incoherent topic drift. Contrastive Decoding (CD), introduced by Li et al. (2022) in *Contrastive Decoding: Open-ended Text Generation as Optimization*, addresses this without any additional training. CD uses a large "expert" model and a small "amateur" model at inference time, selecting tokens that the expert favors but the amateur dislikes, essentially penalizing the failure modes (repetition, incoherence, generic phrasing) that smaller models amplify. Li et al. demonstrate that CD outperforms greedy, top-k, nucleus, typical, and contrastive search decoding on diversity, distributional similarity (MAUVE), and coherence metrics across news, Wikipedia, and story domains.

Chosen Result

We reproduce two main results from Li et al. (2022) to validate the effectiveness of CD. Table 1a, the paper's main benchmark comparison, shows that CD consistently outperforms greedy, nucleus, and contrastive search across Diversity (DIV), MAUVE, and Coherence (COH) metrics using GPT-2 XL and OPT-13B families on news, Wikipedia, and story-generation datasets. Next, Figure 1a shows that DIV and MAUVE improve as the size gap between expert and amateur models increases, with best performance when GPT-2 XL is paired with GPT-2 Small (Li et al., 2022). These results were selected because they establish CD's performance advantages and verify the design choice of maximizing model scale differences. Beyond these text-based findings, we also evaluate the method's generalizability by extending CD to the MusicGen architecture.

Methodology

The core of CD is a decoding objective that scores each candidate token using the difference between the expert (E) and amateur (A) model probabilities:

$$\textit{CD-Score}(x_i) = \log P_E(x_i \mid x_{<i}) - \log P_A(x_i \mid x_{<i})$$

Thus at each decoding step, the model selects tokens favored by the expert while penalizing tokens the amateur also assigns high probability to. To avoid selecting unlikely or nonsensical tokens, Li et al. apply an adaptive plausibility constraint that restricts candidates to tokens whose expert probability is at least a fraction $\alpha=0.1$ of the maximum expert probability. The amateur is also conditioned on the last token of the prefix to encourage generic continuations (Li et al., 2022).

Text Experiments: We reproduce the text-generation setup of Li et al. using GPT-2 XL / Small and OPT-6.7B / 125M expert–amateur pairs (Li et al., 2022). OPT-13B is omitted due to GPU constraints. We additionally evaluate Qwen1.5-7B / 0.5B to test generalization to a more recent architecture. For scaling experiments, we use all GPT-2 size pairs (S/M/L/XL) as in the original paper (Li et al., 2022).

Prompts consist of 1,000 samples of 32-token prefixes drawn from WikiText-103 (Merity et al., 2016), Wikinews (izumi-lab, 2023), and Project Gutenberg (Lahiri, 2014). Each model generates 256-token continuations.

We compare against greedy decoding, top-k sampling (k=50), nucleus sampling (p=0.95), Typical Sampling ($\tau=0.95$) (Meister et al., 2022), and Contrastive Search (k=5, $\alpha=0.6$) (Su & Collier, 2022).

For CD, we use beam size 5, with amateur temperature set to 0.5 (GPT-2) and 1.0 (OPT and Qwen1.5), following Li et al. (2022).

Evaluation uses DIV (product of unique bi/tri/4-gram fractions), COH (cosine similarity between SimCSE prompt and continuation embeddings), and MAUVE (distributional similarity between generated and reference text) (Li et al., 2022; Gao et al., 2021; Pillutla et al., 2021). SimCSE and MAUVE are implemented using their official codebases (Gao et al., 2021 repo; Pillutla et al., 2021 repo).

Music Experiments: We extend Contrastive Decoding to MusicGen (Copet et al., 2023). We use MusicGen-Large as the expert and MusicGen-Small as the amateur. Experiments use GTZAN clips (sanchit-gandhi, 2023), with 5-second audio prompts and 10-second generated continuations. Audio prompts are text-conditioned as "Continue this [genre] piece", where genre labels come from GTZAN metadata.

MusicGen represents audio as four parallel streams of discrete tokens (codebooks) generated at 50 Hz (Copet et al., 2023; Défossez et al., 2022). Earlier codebooks represent coarse audio structure, while later streams capture detail. We evaluate CD by varying (i) amateur text conditioning and (ii) the subset of codebooks to which CD is applied.

Because standard beam search is not directly compatible with MusicGen's codebooks, we replace beam search with top-k sampling (k=50) with the CD objective while keeping all other hyperparameters unchanged. Contrastive Search is also incompatible and therefore omitted.

Evaluation uses PaSST KL-Divergence, which measures divergence between generated and reference audio embedding distributions using PaSST features (Koutini et al., 2021); Fréchet Audio Distance (FAD), which compares generated and real audio distributions in embedding space (gudgud96, 2023 repo); and a VGGish-based adaptation of MAUVE, which measures distributional similarity between generated and reference audio (Pillutla et al., 2021). PaSST and MAUVE were implemented using their official codebases (Koutini et al., 2021 repo; krishnap25, 2021 repo).

Results & Analysis

CD consistently achieves the highest COH across all architectures, confirming Li et al.'s findings (Tables 1a, 1b). Our results deviate on DIV: unlike the original paper, where CD achieved comparable or slightly lower DIV than nucleus/typical, CD dominates DIV in our reproduction across nearly all model-dataset pairs (Tables 1a, 1b). MAUVE fell within 5% of reported values despite using OPT-6.7B instead of OPT-13B. For Qwen1.5-7B, absolute scores are lower across all methods, which is expected for an unaligned base model, but CD still provides the largest gains (Tables 1a, 1b), suggesting the method is effective for newer architectures.

Widening the expert-amateur gap generally improves DIV and MAUVE, with GPT-2 XL / Small yielding near-optimal performance, consistent with Li et al (Figures 1a, 1b). However, our results show significantly less variance across model combinations. We attribute this to our amateur temperature choice (0.5 vs. the likely 1.0 used by the authors). A lower temperature sharpens the amateur's distribution, applying harsher penalties more uniformly and inflating DIV/MAUVE across all size combinations. COH, which was not included in Li et al.'s scaling analysis, peaks along the diagonal where the expert and amateur are identical models (Figure 1b). In this setting, the only difference between the two models is the amateur's truncated context; combined with low-temperature decoding, this could strongly push the expert toward tokens closely related to the full prompt, artificially increasing COH.

CD also transferred effectively to autoregressive audio generation, outperforming all baselines across all metrics (Table 2). The strongest results came from conditioning the amateur on an opposing genre (e.g., reggae expert vs. classical amateur) (Tables 3, 4), possibly because the larger stylistic gap strengthens the contrastive signal. Applying CD only to codebooks 1-3, while leaving codebook 0 unchanged, also outperformed applying CD across all four codebooks (Table 5), suggesting that modifying the most general codebook may disrupt higher-level audio structure and introduce unwanted artifacts. Combining opposing-genre conditioning with codebooks 1-3 produced the best overall results (Table 4), indicating that the two modifications improve different aspects of generation.

Reflections

One key finding was how strongly amateur temperature affected results. Our higher DIV scores relative to Li et al. likely came from using temperature 0.5 instead of the authors' unreported setting, likely 1.0. Even with a faithful implementation, small hyperparameter differences can substantially change behavior, highlighting a common reproducibility issue in NLP. Future reproductions should include sensitivity analyses for key decoding parameters.

Extending CD to MusicGen also required several design changes. Beam search was incompatible with MusicGen's decoding architecture, so we replaced it with top-k sampling over CD scores. MusicGen's hierarchical token streams also introduced the question of where to apply CD. Applying CD to codebooks 1-3 improved performance, while modifying codebook 0 degraded it, suggesting that perturbing coarse structural representations introduces artifacts and that CD is more effective on finer detail.

Opposing-genre amateur prompting also improved results beyond simply suppressing generic output. This may generalize to text generation: conditioning the amateur on an opposing topic, style, or persona could sharpen expert outputs along a target dimension.

Finally, CD's scaling behavior in audio remains unexplored. Its interaction with instruction-tuned or RLHF-aligned models also warrants study, since the amateur's failure modes may differ from those of a base model. Our Qwen1.5 results suggest CD may provide a lightweight quality improvement for newer open-weight base models.

References

Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., & Défossez, A. (2023). Simple and controllable music generation. arXiv preprint arXiv:2306.05284. https://arxiv.org/abs/2306.05284

Défossez, A., Copet, J., Synnaeve, G., & Adi, Y. (2022). High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438*. https://arxiv.org/abs/2210.13438

Gao, T., Yao, X., & Chen, D. (2021). SimCSE: Simple contrastive learning of sentence embeddings. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. https://doi.org/10.18653/v1/2021.emnlp-main.552

Gao, T., Yao, X., & Chen, D. (2021). SimCSE: Simple contrastive learning of sentence embeddings [Source code]. GitHub. https://github.com/princeton-nlp/SimCSE

gudgud96. (2023). Frechet-audio-distance [Source code]. GitHub. https://github.com/gudgud96/frechet-audio-distance

izumi-lab. (2023). Wikinews-en-20230728 [Dataset]. Hugging Face. https://huggingface.co/datasets/izumi-lab/wikinews-en-20230728

Koutini, K., Schlüter, J., Eghbal-zadeh, H., & Widmer, G. (2021). Efficient training of audio transformers with patchout. *arXiv preprint arXiv:2110.05069*. https://arxiv.org/abs/2110.05069

Koutini, K., Schlüter, J., Eghbal-zadeh, H., & Widmer, G. (2021). PaSST: Patchout audio spectrogram transformer [Source code]. GitHub. https://github.com/kkoutini/passt_hear21

krishnap25. (2021). mauve [Source code]. GitHub. https://github.com/krishnap25/mauve

Lahiri, S. (2014). Project Gutenberg dataset [Dataset]. https://shibamoulilahiri.github.io/gutenberg_dataset.html

Li, X. L., Holtzman, A., Fried, D., Liang, P., Weston, J., Zettlemoyer, L., Lewis, M., & Hajishirzi, H. (2022). Contrastive decoding: Open-ended text generation as optimization. arXiv preprint arXiv:2210.15097. https://arxiv.org/abs/2210.15097

Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). WikiText long term dependency language modeling dataset [Dataset]. Hugging Face. https://huggingface.co/datasets/Salesforce/wikitext

Meister, C., Pimentel, T., Wiher, G., & Cotterell, R. (2022). Locally typical sampling. arXiv preprint arXiv:2202.00666. https://arxiv.org/abs/2202.00666

Pillutla, K., Swayamdipta, S., Zellers, R., Thickstun, J., Welleck, S., Choi, Y., & Harchaoui, Z. (2021). MAUVE: Measuring the gap between neural text and human text using divergence frontiers. *arXiv*. https://doi.org/10.48550/arxiv.2102.01454

sanchit-gandhi. (2023). gtzan [Dataset]. Hugging Face. https://huggingface.co/datasets/sanchit-gandhi/gtzan

Su, Y., & Collier, N. (2022). Contrastive search is what you need for neural text generation. *arXiv*. https://doi.org/10.48550/arxiv.2210.14140

Appendix

A.1 Text Generation Reproduction

		wikinews			wikitext			story		
	name	DIV	MAUVE	COH	DIV	MAUVE	COH	DIV	MAUVE	COH
OPT-13B	max prob	0.08	0.3	0.65	0.03	0.08	0.63	0.02	0.05	0.51
	k=50	0.91	0.92	0.64	0.72	0.77	0.64	0.91	0.9	0.51
	p=0.95	0.92	0.92	0.62	**0.92**	0.89	0.55	0.93	0.91	0.48
	typical=0.95	**0.94**	0.9	0.59	0.89	0.86	0.58	**0.95**	0.91	0.46
	CS(Su et al., 2022)	0.92	0.87	0.59	0.87	0.77	0.52	0.81	0.78	0.47
	CD	**0.94**	**0.94**	**0.69**	0.91	**0.91**	**0.69**	0.89	**0.94**	**0.62**
GPT2-XL	max prob	0.04	0.14	0.65	0.02	0.05	0.62	0.01	0.03	0.49
	k=50	0.92	0.88	0.64	0.87	0.79	0.61	0.91	0.87	0.51
	p=0.95	0.94	0.9	0.6	0.92	0.87	0.57	0.94	0.91	0.46
	typical=0.95	**0.95**	0.91	0.56	**0.95**	0.84	0.53	**0.96**	0.88	0.43
	CS(Su et al., 2022)	0.93	0.82	0.62	0.86	0.75	0.59	0.88	0.78	0.48
	CD	0.92	**0.94**	**0.69**	0.89	**0.92**	**0.69**	0.83	**0.94**	**0.64**

Table 1a: Original baseline evaluation results for Wikinews, WikiText, and Story datasets (Li et al., 2022). The best scores for each model and domain are in bold.

		wikinews			wikitext			story		
Model	Method	DIV	MAUVE	COH	DIV	MAUVE	COH	DIV	MAUVE	COH
GPT2-XL	Greedy	0.030	0.112	0.643	0.014	0.038	0.615	0.005	0.022	0.438
	Top-k	0.864	0.881	0.640	0.795	0.776	0.612	0.842	0.900	0.480
	Nucleus	0.843	0.888	0.646	0.741	0.741	0.625	0.794	0.870	0.480
	Typical	0.839	0.851	0.648	0.746	0.751	0.627	0.803	0.875	0.489
	CS	0.880	0.861	0.631	0.734	0.663	0.605	0.768	**0.906**	0.445
	CD	**0.933**	**0.939**	**0.688**	**0.895**	**0.939**	**0.691**	**0.949**	0.877	**0.581**
OPT-6.7B	Greedy	0.051	0.143	0.656	0.024	0.063	0.630	0.011	0.037	0.483
	Top-k	0.844	**0.929**	0.649	0.795	0.842	0.630	0.828	0.894	0.486
	Nucleus	0.825	0.849	0.660	0.778	0.775	0.637	0.783	**0.895**	0.489
	Typical	0.828	0.879	0.660	0.771	0.801	0.637	0.778	0.836	0.495
	CS	0.873	0.823	0.634	0.805	0.758	0.604	0.764	0.787	0.451
	CD	**0.905**	0.900	**0.688**	**0.855**	**0.928**	**0.691**	**0.865**	0.892	**0.594**
Qwen1.5-7B	Greedy	0.133	0.405	0.655	0.319	0.600	0.695	0.025	0.098	0.466
	Top-k	0.794	**0.876**	0.650	0.623	0.801	0.682	0.841	**0.884**	0.485
	Nucleus	0.755	0.872	0.659	0.586	0.783	0.695	0.797	0.858	0.493
	Typical	0.753	0.853	0.653	0.583	0.779	0.692	0.794	0.881	0.488
	CS	0.727	0.799	0.643	0.584	0.724	**0.716**	0.604	0.703	0.465
	CD	**0.885**	0.693	**0.668**	**0.778**	**0.852**	0.713	**0.913**	0.756	**0.527**

Table 1b: Our reproduction of baseline evaluation results across Wikinews, WikiText, and Story datasets. The best scores for each model and domain are in bold.

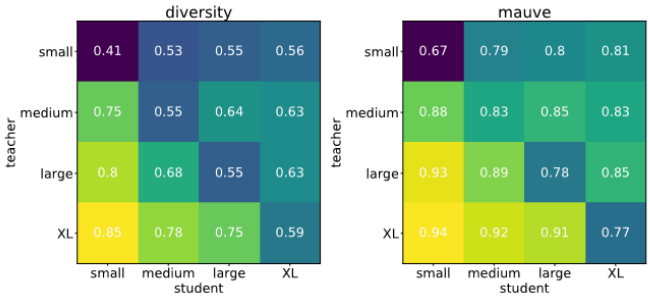

Figure 1a: Original scaling experiments demonstrating generation quality across varying sizes of GPT-2 expert (teacher) and amateur (student) models (Li et al., 2022). Note: The original study did not report Coherence scaling.

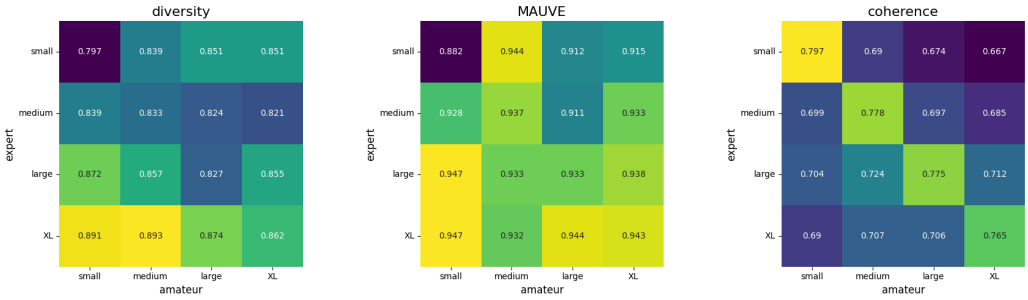

Figure 1b: Our reproduction of the scaling experiments. Note the significantly reduced variance in Diversity and MAUVE compared to the original study, and the diagonal peak in Coherence.

A.2 Music Generation Extension

Decoding Method	PaSST KL (↓)	FAD (↓)	MAUVE (↑)
Greedy	0.508	6.910	0.035
Top-k	0.086	1.282	0.484
Nucleus	0.114	1.940	0.328
Typical	0.113	1.874	0.321
CD	**0.080**	**1.103**	**0.499**

Table 2: MusicGen decoding method comparison on the GTZAN dataset. Contrastive Decoding outperforms all baselines. (↓ lower is better; ↑ higher is better). The best scores for each metric are in bold.

Target Genre	Anti-Genre (Amateur Prompt)
blues	math rock
classical	dubstep
country	ambient lofi
disco	funeral march
hiphop	classical symphony
jazz	heavy metal
metal	smooth jazz
pop	dissonant avant-garde
reggae	classical piano
rock	lullaby

Table 3: Target and anti-genre text prompt pairings used for adversarial amateur prompting.

Text Prompting Strategy	PaSST KL (↓)	FAD (↓)	MAUVE (↑)
Both models get genre prompt	0.080	1.103	0.499
Amateur no prompt	0.143	2.171	0.270
Amateur opposite genre prompt	0.071	1.074	0.518
Amateur opposite genre prompt + codebooks 1, 2, 3	**0.069**	**1.044**	**0.521**

Table 4: Comparison of amateur text prompting strategies. Providing the amateur with an opposing genre prompt yields the strongest alignment with the target genre. The best scores for each metric are in bold.

CD Application Level	**PaSST KL** (↓)	**FAD** (↓)	**MAUVE** (↑)
CD on codebook 0	0.080	1.172	0.484
CD on codebook 1	0.081	1.217	0.474
CD on codebook 2	0.080	1.193	0.485
CD on codebook 3	0.079	1.258	0.465
CD on codebooks 1, 2, 3	**0.077**	1.128	**0.506**
CD on all codebooks	0.080	**1.103**	0.499

Table 5: Performance of Contrastive Decoding applied across different EnCodec codebook levels. Applying CD to finer-grained codebooks (1-3) outperforms application to all codebooks in PaSST KL and MAUVE.